

MAGNO: Multiscale Attentional Graph Neural Operator

1 Reference

The Multiscale Attentional Graph Neural Operator (MAGNO) is introduced in the Geometry Aware Operator Transformer (GAOT) framework for learning PDE solution operators on arbitrary domains.

S. Wen, A. Kumbhat, L. Lingsch, S. Mousavi, Y. Zhao, P. Chandrashekar, and S. Mishra, “Geometry Aware Operator Transformer as an Efficient and Accurate Neural Surrogate for PDEs on Arbitrary Domains,” NeurIPS 2025.

arXiv: [2505.18781](#).

2 Introduction

The Multiscale Attentional Graph Neural Operator (MAGNO) is a geometry-aware neural operator designed for PDEs defined on arbitrary domains represented by point clouds.

MAGNO extends the Graph Neural Operator (GNO) by combining:

1. local graph-based kernel integration,
2. multiple spatial scales,
3. attention-based quadrature weights,
4. learned geometry embeddings,
5. adaptive combination of information from different scales.

The principal idea is that a single neighborhood radius may be insufficient for PDEs containing structures at multiple spatial scales. MAGNO therefore evaluates local operator interactions using several neighborhood radii and learns how strongly each scale should contribute.

3 Graph Neural Operator Background

Let the input point cloud be

$$D_{\Delta} = \{x_k\}_{k=1}^N, \tag{1}$$

where each point x_k has an associated input feature

$$a(x_k). \tag{2}$$

Let y denote a point in the latent point cloud.

A standard GNO performs a local kernel aggregation,

$$w_e(y) = \sum_{k=1}^{n_y} \alpha_k K(y, x_k, a(x_k)) \varphi(a(x_k)), \quad (3)$$

where the sum is restricted to neighboring points satisfying

$$|y - x_k| \leq r. \quad (4)$$

Here,

- K is a learned kernel,
- φ is a learned feature transformation,
- α_k are quadrature weights,
- r is the neighborhood radius.

The limitation is that a single radius r provides only one spatial scale.

4 Multiscale Construction

MAGNO introduces a collection of spatial scales.

Let

$$r_m = s_m r_0, \quad m = 1, \dots, M, \quad (5)$$

where

- r_0 is a base radius,
- s_m is the scale factor,
- r_m is the radius at scale m .

For each scale m , define the local neighborhood

$$\mathcal{N}_m(y) = \{x_k \in D_\Delta : |y - x_k| \leq r_m\}. \quad (6)$$

Let

$$n_y^m = |\mathcal{N}_m(y)|. \quad (7)$$

The scale-specific graph operator is

$$\boxed{\tilde{w}_e^m(y) = \sum_{k=1}^{n_y^m} \alpha_k^m K^m(y, x_k, a(x_k)) \varphi(a(x_k))}. \quad (8)$$

This is the fundamental local operator used at scale m .

5 Attention-Based Quadrature Weights

Instead of using fixed quadrature weights, MAGNO introduces attention-based weights.

For each neighboring point x_k ,

$$e_k^m = \frac{\langle W_q^m y, W_\kappa^m x_k \rangle}{\sqrt{d}}. \quad (9)$$

Here,

$$W_q^m \quad (10)$$

is a learned query projection and

$$W_\kappa^m \quad (11)$$

is a learned key projection.

The normalized attention weight is

$$\alpha_k^m = \frac{\exp(e_k^m)}{\sum_{k'=1}^{n_y^m} \exp(e_{k'}^m)}. \quad (12)$$

Consequently,

$$\sum_{k=1}^{n_y^m} \alpha_k^m = 1. \quad (13)$$

The attention mechanism allows MAGNO to learn which neighboring points are important for the representation at y .

6 Scale-Specific AGNO

The combination of the local kernel integral and attention weights defines an Attentional Graph Neural Operator (AGNO) at scale m :

$$\tilde{w}_e^m(y) = \sum_{x_k \in \mathcal{N}_m(y)} \alpha_k^m K^m(y, x_k, a(x_k)) \varphi(a(x_k)). \quad (14)$$

Thus each scale independently produces a representation

$$\tilde{w}_e^m(y). \quad (15)$$

7 Geometry Embeddings

Coordinates alone may not fully describe the local geometry of an arbitrary domain.

For each point y and scale m , MAGNO can construct local geometric descriptors from the neighborhood

$$\mathcal{N}_m(y). \quad (16)$$

Examples include the number of neighboring points,

$$n_y^m, \quad (17)$$

the average distance,

$$D_{\text{avg}}^m(y) = \frac{1}{n_y^m} \sum_{k=1}^{n_y^m} |x_k - y|, \quad (18)$$

and the distance variance,

$$D_{\text{var}}^m(y) = \frac{1}{n_y^m} \sum_{k=1}^{n_y^m} (|x_k - y| - D_{\text{avg}}^m(y))^2. \quad (19)$$

The centroid offset is

$$\Delta_y^m = \frac{1}{n_y^m} \sum_{k=1}^{n_y^m} (x_k - y). \quad (20)$$

Additional PCA-based descriptors can be obtained from the covariance matrix of the neighboring offsets.

These geometric descriptors are concatenated into

$$z_y^m = \text{concat} (n_y^m, D_{\text{avg}}^m, D_{\text{var}}^m, \Delta_y^m, \text{PCA}^m(y)). \quad (21)$$

A geometry-embedding network then produces

$$g^m(y) = \text{MLP}_g(z_y^m). \quad (22)$$

8 Multiscale Attention

The scale-specific representations must now be combined.

MAGNO does not simply sum the different scales. Instead, it learns a scale-dependent weighting.

Let

$$\psi_m(y) \quad (23)$$

be a learned scalar produced by a small MLP.

The scale attention coefficients are

$$\beta_m(y) = \frac{\exp(\psi_m(y))}{\sum_{m'=1}^M \exp(\psi_{m'}(y))}. \quad (24)$$

Therefore,

$$\sum_{m=1}^M \beta_m(y) = 1. \quad (25)$$

The final multiscale representation is

$$w_e(y) = \sum_{m=1}^M \beta_m(y) \hat{w}^m(y). \quad (26)$$

Here $\widehat{w}^m(y)$ denotes the scale-specific representation, possibly after incorporating the corresponding geometry embedding.

Thus MAGNO adaptively determines the relative importance of different spatial scales.

9 Complete MAGNO Encoder

The MAGNO encoder can therefore be summarized as

$$a(x_k) \longrightarrow \varphi(a(x_k)), \quad (27)$$

followed by multiscale neighborhood construction,

$$r_m = s_m r_0, \quad (28)$$

and scale-specific attention,

$$\alpha_k^m = \text{softmax}_k \left(\frac{\langle W_q^m y, W_\kappa^m x_k \rangle}{\sqrt{d}} \right). \quad (29)$$

Each scale produces

$$\tilde{w}_e^m(y) = \sum_{x_k \in \mathcal{N}_m(y)} \alpha_k^m K^m(y, x_k, a(x_k)) \varphi(a(x_k)). \quad (30)$$

Finally,

$$w_e(y) = \sum_{m=1}^M \beta_m(y) \widehat{w}^m(y). \quad (31)$$

10 MAGNO in GAOT

GAOT follows an encode–process–decode architecture:

$$\boxed{\text{Input} \rightarrow \text{MAGNO Encoder} \rightarrow \text{Transformer Processor} \rightarrow \text{MAGNO Decoder} \rightarrow \text{Output}}. \quad (32)$$

The encoder maps physical point-cloud data to geometry-aware latent tokens.

The transformer processor performs global interactions between the latent tokens.

The MAGNO decoder then maps the latent representation back to arbitrary query points.

11 MAGNO Decoder

For a query point x , the decoder identifies neighboring latent tokens and applies the same multiscale graph-operator construction.

At scale m ,

$$\tilde{w}_d^m(x) = \sum_{y_k \in \mathcal{N}_m(x)} \alpha_k^m K_d^m(x, y_k, w(y_k)) \varphi_d(w(y_k)). \quad (33)$$

The final decoded representation is

$$w_d(x) = \sum_{m=1}^M \beta_m(x) \widehat{w}_d^m(x). \quad (34)$$

A final projection produces the predicted physical field,

$$\hat{u}(x) = Q(w_d(x)). \quad (35)$$

12 Overall Mathematical Formulation

The complete MAGNO mechanism can be summarized as

$$\tilde{w}^m(y) = \sum_{x_k \in \mathcal{N}_m(y)} \alpha_k^m K^m(y, x_k, a(x_k)) \varphi(a(x_k)). \quad (36)$$

The local attention is

$$\alpha_k^m = \frac{\exp\left(\frac{\langle W_q^m y, W_\kappa^m x_k \rangle}{\sqrt{d}}\right)}{\sum_{k' \in \mathcal{N}_m(y)} \exp\left(\frac{\langle W_q^m y, W_\kappa^m x_{k'} \rangle}{\sqrt{d}}\right)}. \quad (37)$$

The scale attention is

$$\beta_m(y) = \frac{\exp(\psi_m(y))}{\sum_{m'=1}^M \exp(\psi_{m'}(y))}. \quad (38)$$

The final multiscale aggregation is

$$w_e(y) = \sum_{m=1}^M \beta_m(y) \tilde{w}^m(y). \quad (39)$$

13 PyTorch Implementation

```

1
2 import torch
3 import torch.nn as nn
4 import torch.nn.functional as F
5
6
7 class AttentionKernel(nn.Module):
8
9     def __init__(
10         self,
11         coord_dim,
12         feature_dim,
13         hidden_dim
14     ):
15         super().__init__()
16
17         self.query = nn.Linear(
18             coord_dim,
19             hidden_dim
20         )
21

```

```

22     self.key = nn.Linear(
23         coord_dim,
24         hidden_dim
25     )
26
27     self.kernel = nn.Sequential(
28         nn.Linear(
29             2 * coord_dim + feature_dim,
30             hidden_dim
31         ),
32         nn.GELU(),
33         nn.Linear(
34             hidden_dim,
35             feature_dim
36         )
37     )
38
39     self.value = nn.Sequential(
40         nn.Linear(
41             feature_dim,
42             hidden_dim
43         ),
44         nn.GELU(),
45         nn.Linear(
46             hidden_dim,
47             feature_dim
48         )
49     )
50
51     self.scale = hidden_dim ** 0.5
52
53     def forward(
54         self,
55         query_coords,
56         source_coords,
57         source_features,
58         radius
59     ):
60
61         B, Ny, _ = query_coords.shape
62         Nx = source_coords.shape[1]
63
64         q = self.query(query_coords)
65         k = self.key(source_coords)
66
67         scores = torch.einsum(
68             "bqd,bkd->bqk",
69             q,
70             k
71         ) / self.scale
72
73         distances = torch.cdist(
74             query_coords,
75             source_coords
76         )
77
78         mask = distances <= radius
79

```

```

80         scores = scores.masked_fill(
81             ~mask,
82             float("-inf")
83         )
84
85         attention = F.softmax(
86             scores,
87             dim=-1
88         )
89
90         values = self.value(
91             source_features
92         )
93
94         aggregated = torch.einsum(
95             "bqk,bkc->bqc",
96             attention,
97             values
98         )
99
100         return aggregated
101
102
103 class AGNO(nn.Module):
104
105     def __init__(
106         self,
107         coord_dim,
108         feature_dim,
109         hidden_dim,
110         radius
111     ):
112         super().__init__()
113
114         self.radius = radius
115
116         self.operator = AttentionKernel(
117             coord_dim=coord_dim,
118             feature_dim=feature_dim,
119             hidden_dim=hidden_dim
120         )
121
122         self.projection = nn.Linear(
123             feature_dim,
124             feature_dim
125         )
126
127     def forward(
128         self,
129         query_coords,
130         source_coords,
131         source_features
132     ):
133
134         out = self.operator(
135             query_coords,
136             source_coords,
137             source_features,

```



```

138         self.radius
139     )
140
141     return self.projection(out)
142
143
144 class MAGNO(nn.Module):
145
146     def __init__(
147         self,
148         coord_dim,
149         in_channels,
150         out_channels,
151         width=64,
152         hidden_dim=64,
153         radii=(0.1, 0.2, 0.4)
154     ):
155         super().__init__()
156
157         self.radii = radii
158         self.num_scales = len(radii)
159
160         self.input_projection = nn.Linear(
161             in_channels + coord_dim,
162             width
163         )
164
165         self.agno_layers = nn.ModuleList(
166             [
167                 AGNO(
168                     coord_dim=coord_dim,
169                     feature_dim=width,
170                     hidden_dim=hidden_dim,
171                     radius=radius
172                 )
173                 for radius in radii
174             ]
175         )
176
177         self.scale_attention = nn.Sequential(
178             nn.Linear(
179                 coord_dim,
180                 hidden_dim
181             ),
182             nn.GELU(),
183             nn.Linear(
184                 hidden_dim,
185                 self.num_scales
186             )
187         )
188
189         self.output_projection = nn.Sequential(
190             nn.Linear(
191                 width,
192                 hidden_dim
193             ),
194             nn.GELU(),
195             nn.Linear(

```

```

196         hidden_dim,
197         out_channels
198     )
199 )
200
201 def forward(
202     self,
203     coords,
204     features
205 ):
206
207     x = torch.cat(
208         [
209             coords,
210             features
211         ],
212         dim=-1
213     )
214
215     x = self.input_projection(x)
216
217     scale_outputs = []
218
219     for agno in self.agno_layers:
220
221         scale_output = agno(
222             query_coords=coords,
223             source_coords=coords,
224             source_features=x
225         )
226
227         scale_outputs.append(
228             scale_output
229         )
230
231     scale_outputs = torch.stack(
232         scale_outputs,
233         dim=-2
234     )
235
236     scale_logits = self.scale_attention(
237         coords
238     )
239
240     scale_weights = F.softmax(
241         scale_logits,
242         dim=-1
243     )
244
245     scale_weights = scale_weights.unsqueeze(
246         -1
247     )
248
249     x = (
250         scale_outputs
251         * scale_weights
252     ).sum(
253         dim=-2

```

```

254         )
255
256         return self.output_projection(x)
257
258
259 if __name__ == "__main__":
260
261     device = (
262         "cuda"
263         if torch.cuda.is_available()
264         else "cpu"
265     )
266
267     batch_size = 2
268     n_points = 256
269
270     coords = torch.rand(
271         batch_size,
272         n_points,
273         2,
274         device=device
275     )
276
277     features = torch.randn(
278         batch_size,
279         n_points,
280         1,
281         device=device
282     )
283
284     model = MAGNO(
285         coord_dim=2,
286         in_channels=1,
287         out_channels=1,
288         width=64,
289         hidden_dim=64,
290         radii=(0.1, 0.2, 0.4)
291     ).to(device)
292
293     prediction = model(
294         coords,
295         features
296     )
297
298     print(
299         "Input:",
300         features.shape
301     )
302
303     print(
304         "Prediction:",
305         prediction.shape
306     )

```

14 Training

```

1  optimizer = torch.optim.Adam(
2      model.parameters(),
3      lr=1e-3
4  )
5
6
7  criterion = nn.MSELoss()
8
9  for epoch in range(100):
10
11      model.train()
12
13      optimizer.zero_grad()
14
15      prediction = model(
16          train_coords,
17          train_input
18      )
19
20      loss = criterion(
21          prediction,
22          train_target
23      )
24
25      loss.backward()
26
27      optimizer.step()
28
29      if epoch % 10 == 0:
30
31          print(
32              f"Epoch {epoch}: "
33              f"Loss = {loss.item():.6e}"
34          )

```

15 Summary

MAGNO extends the Graph Neural Operator by introducing multiple spatial scales and learned attention over both neighboring points and scales.

The scale-specific operator is

$$\tilde{w}^m(y) = \sum_{x_k \in \mathcal{N}_m(y)} \alpha_k^m K^m(y, x_k, a(x_k)) \varphi(a(x_k)). \quad (40)$$

The point-level attention is

$$\alpha_k^m = \text{softmax}_k \left(\frac{\langle W_q^m y, W_\kappa^m x_k \rangle}{\sqrt{d}} \right). \quad (41)$$

The scale-level attention is

$$\beta_m(y) = \text{softmax}_m (\psi_m(y)). \quad (42)$$

The final multiscale operator is

$$\boxed{w_e(y) = \sum_{m=1}^M \beta_m(y) \hat{w}^m(y)}. \quad (43)$$

Hence the central idea of MAGNO is

$$\boxed{\text{Local Graph Operator} + \text{Point Attention} + \text{Multiple Spatial Scales} + \text{Scale Attention} + \text{Geometry Info}}. \quad (44)$$